

Lost and Found Platform (Sajha Khoj)

Community Web Platform — Step-by-Step Viva & Defense Preparation Guide

1. How to Introduce Your Project (1-Minute Opening Pitch)

■ 60-Second Perfect Viva Introduction

Examiner Prompt: 'Tell me about your project.'

*'Respected examiner, our project is **Sajha Khoj** (Community Lost & Found Platform), a production-ready full-stack web application built using the **MERN Stack (MongoDB, Express, React, Node.js)**, Vite, Cloudinary, and Nodemailer.*

*Traditional platforms like Facebook groups and bulletin boards suffer from high spam, privacy risks, and false claims. Sajha Khoj solves this fundamental trust issue through our signature **Double-Handshake Verification Workflow** — where claimants must submit private proof of ownership, finders review and approve the claim, and only verified parties exchange contact details.*

*The platform also integrates a **gamified reputation system (Hero Points & Badges)**, an **intelligent Next Step Portal**, multi-parameter filtering, daily quota rate-limiting, and a dedicated **Admin Moderation Dashboard**.'*

2. System Architecture & Technology Stack Breakdown

The system follows a 3-tier decoupled Client-Server architecture with secure RESTful APIs.

Layer / Area	Technology & Version	Role in Project & Technical Reason for Selection
Frontend Framework	React.js 18.2 + Vite 4.2	Component-based Single Page App (SPA). Vite provides sub-second HMR and instant builds compared to Webpack.
Client Routing	React Router DOM 6.10	Client-side routing with protected route guards for Authenticated & Admin-only views.
Styling & Animation	Tailwind CSS + Framer Motion	Utility-first responsive layouts with smooth micro-animations and page transitions.
State & HTTP	Context API + Axios 1.3	Global Auth state management with Axios request interceptors automatically attaching JWT Bearer tokens.
Backend Server	Node.js 18+ & Express.js 4.18	Fast, event-driven, non-blocking REST API server organized into clean MVC controllers & routes.
Database & ODM	MongoDB Atlas + Mongoose 7	Document-based NoSQL database. Embedded sub-document schema handles claims and rewards with high read speed.
Authentication	JWT + bcryptjs (10 rounds)	Stateless token-based authentication and secure salted one-way password hashing.
Image Cloud Storage	Multer 1.4 + Cloudinary SDK	In-memory buffer upload streamed to Cloudinary CDN; MongoDB stores only the secure image URL.
Email Gateway	Nodemailer + Gmail SMTP	Automated delivery of account activation links and time-limited password reset tokens.
API Security	Helmet + CORS + Rate Limiters	Sets secure HTTP headers, restricts origin domains, and prevents brute-force / DDoS attacks.
Data Visualization	Recharts 3.8	Renders administrative analytics for monthly item trends, category distribution, and recovery rates.

3. Core Innovation: Double-Handshake Verification Workflow

Examiners always ask about the core business logic. Explain this 6-step lifecycle clearly:

- **Step 1: Item Listing Creation** — A user posts a Lost or Found item with title, category, description, location, date, and image. Found items can specify optional reward details. The backend verifies daily posting limits (12 items/day for free users).
- **Step 2: Proof Submission (Claim)** — A claimant views the item and submits a private claim detailing non-public proof of ownership (e.g. engravings, invoice number, phone lock screen) and contact info.
- **Step 3: Finder Verification & Approval** — The finder reviews incoming claims in their dashboard. When they click 'Approve', the backend marks that claim as 'approved' and automatically marks competing pending claims as 'rejected'.
- **Step 4: Mutual Contact Reveal & Handover** — Only upon approval are phone numbers and emails unlocked so finder and claimant can coordinate a safe meetup.
- **Step 5: Owner Final Confirmation** — Once the owner physically receives the item, they click 'Confirm Recovery' (`/confirm-recovery`). The item status updates to 'resolved'.
- **Step 6: Gamification & Badge Reward** — The finder receives **+500 Hero Points**. The user model evaluates `checkAndAwardBadges()`, awarding tiered badges and triggering in-app celebration notifications.

■ Viva Defense Advantage

Why is this better than typical CRUD apps? Most student projects use simple single-button deletes or status toggles. Sajha Khoj models real-world multi-party verification with state machines, role enforcement, and gamified incentives.

4. Database Schema Design & Relationships

MongoDB NoSQL design balances relational references with high-performance embedded subdocuments:

1. User Collection (`User.js`):

- name, email (unique, lowercase), password (bcrypt hash), phone, role (`['user', 'admin']`).
- reputationPoints (Number), rating (0-10), totalResolved (Number), badges (Array of subdocs: type, name, earnedAt).
- plan (`['free', 'premium']`), itemsCreatedToday, itemsCreatedDate, planRequest (status, message, adminResponse).
- isVerified (Boolean), verificationToken, resetToken, resetTokenExpiry.

2. Item Collection (`Item.js`):

- type (`['lost', 'found']`), title, description, category, location, date, image (Cloudinary URL).
- poster (ObjectId ref -> User), status (`['active', 'resolved']`), returnedBy (ref -> User), confirmedByOwner (Boolean).
- claims (Embedded Sub-document Array): user (ref), message, phone, email, status (`['pending', 'approved', 'rejected']`).
- reward (Embedded Sub-document): amount, currency ('NPR'), description, claimed (Boolean), claimedBy (ref).

3. Notification & Support Report Collections:

- Notification.js: recipient, sender, item, type ('claim', 'approval', 'reward', 'badge', 'handshake'), message, isRead.
- Report.js: subject, category, message, userId, status ('pending', 'resolved').

5. Gamification & Reputation Engine Details

Badge Name	Tier	Items Resolved	Badge Significance
First Find	Bronze	1 item	Welcoming milestone for new community finders.
Rising Hero	Bronze	5 items	Demonstrates active participation and reliable returns.
Community Helper	Silver	10 items	Double-digit verified item returns.
Trusted Finder	Silver	25 items	High credibility marker across the local area.
Hero Award	Gold	50 items	Core community pillar and high-reputation citizen.
Super Hero	Gold	100 items	Centurion resolver with master-level trust.
Legend	Platinum	200 items	Top-tier hall-of-fame community contributor.
Master	Diamond	500 items	Supreme platform master.

6. Security, Rate Limiting & Data Privacy Architecture

- **Password Encryption (bcryptjs):** Salts (10 rounds) and one-way hashes prevent exposure even in case of database leakage.
- **Stateless JWT Bearer Auth:** Tokens signed with a secret key expire automatically; validated via backend protect middleware.
- **Private Proof Shielding:** Contact information and sensitive proof messages are hidden until the finder explicitly approves a claim.
- **Multi-Tier Rate Limiting:** express-rate-limit shields authentication routes against brute-force attacks and item creation against flooding.
- **Daily Quota Enforcement:** User schema tracks itemsCreatedToday and automatically resets at midnight to prevent bot spam.
- **HTTP Protection (Helmet + CORS):** Secures against XSS, clickjacking, MIME sniffing, and cross-origin domain forging.

7. Master Viva Questions & Model Answers (Categorized)

Study these categorized questions thoroughly. They represent 95%+ of questions asked by external examiners.

1. What problem does Sajha Khoj solve in society?

Physical lost-and-found desks are localized and inefficient. Social media posts get lost in algorithm feeds and expose owners to scammers. Sajha Khoj provides a centralized, searchable database with private proof verification to ensure lost goods reach their rightful owners safely.

2. Who are the primary stakeholders / users of this system?

There are 3 main user groups: (1) Finders who discover lost items and want to safely return them; (2) Claimants/Owners seeking lost possessions; (3) Platform Admins who oversee moderation, verify users, and manage plan upgrades.

3. What is the significance of the Nepali name 'Sajha Khoj'?

'Sajha Khoj' translates to 'Shared Search' (साझा खोज), reflecting our core mission of collaborative community search and civic responsibility in Nepal.

4. Why React 18 for the user interface?

React provides component reusability, virtual DOM for high rendering performance, declarative UI state, and seamless integration with libraries like Framer Motion for animations and Recharts for analytics.

5. How does React's Virtual DOM improve performance?

When state changes, React creates a new Virtual DOM tree, calculates the minimal difference (diffing algorithm) against the previous tree, and batches updates to the real DOM (reconciliation), preventing expensive full-page re-renders.

6. Explain how AuthContext works in your frontend.

`AuthContext.jsx` wraps the entire application using React's Context API. It stores the active `user` object and `token`. When a user logs in, `login(token, userData)` stores the token in `localStorage` and updates global state; all components re-render accordingly.

7. What is an Axios Interceptor and why is it used?

In `src/api.js`, an Axios request interceptor intercepts every outgoing HTTP request and automatically attaches `Authorization: Bearer` from `localStorage`, ensuring authenticated endpoints receive tokens without repeating code in every component.

8. How are Protected Routes implemented in React Router v6?

`PrivateRoute` checks if `user` is logged in. If authenticated, it renders `{} or the child component; otherwise, it redirects to /login. For Admin routes, it also verifies user.role === 'admin'.`

9. What is the role of the 'Next Step Portal' component?

`NextStepPortal.jsx` is a floating smart action hub. It evaluates pending items, unapproved claims, and actionable notifications, prompting the user with one-click navigation to their most urgent task.

10. Why Node.js and Express for the backend?

Node.js uses a single-threaded, event-driven, non-blocking asynchronous I/O model based on the V8 engine, making it lightweight and highly scalable for I/O-heavy web services like REST APIs.

11. Explain the MVC (Model-View-Controller) architecture in your backend.

Our backend separates concerns into: (1) Models (`models/*.js` - schema & DB rules), (2) Controllers (`controllers/*.js` - business logic), (3) Routes (`routes/*.js` - URL routing & middleware binding). The View is decoupled and served by React.

12. What is Middleware in Express? Name three you used.

Middleware functions intercept incoming HTTP requests before reaching the controller. Examples: `protect` (verifies JWT), `isAdmin` (checks admin role), `upload.single('image')` (Multer file parser), and `express-validator` (sanitizes inputs).

13. How does Multer handle image uploads without filling server disk space?

We use `multer.memoryStorage()`, which keeps the incoming image buffer temporarily in RAM instead of writing to disk. The buffer is immediately streamed to Cloudinary CDN via `upload_stream`, keeping the backend server stateless and clean.

14. How does password hashing with bcryptjs prevent rainbow table attacks?

Bcrypt uses a work-factor salt (10 rounds in our app). Each salt is uniquely generated, meaning identical passwords produce completely different hash outputs, making pre-computed rainbow table dictionaries useless.

15. Why did you choose MongoDB over a SQL database like PostgreSQL/MySQL?

MongoDB's JSON-like document structure naturally models nested items, claim histories, and reward objects as embedded arrays. It offers schema flexibility, fast write performance, and painless cloud scaling with MongoDB Atlas.

16. When do you use Embedding vs. Referencing in MongoDB?

We embed `claims` inside `Item` because claims are always queried with the item and have an 1-to-few relationship. We reference `poster` (User ObjectId) because users exist independently and have extensive personal profiles.

17. What is the purpose of Mongoose `.populate()`?

`.populate('poster', 'name email phone')` acts like a SQL JOIN, dynamically replacing the stored User ObjectId with the actual matching user document's selected fields during query execution.

18. What are Mongoose Schema Pre-save Hooks?

``userSchema.pre('save', ...)`` runs automatically before saving a document. We use it to hash modified passwords and check badge qualification thresholds before committing changes.

19. How does JWT authentication work from start to finish?

(1) User sends email/password to ``/api/auth/login``; (2) Server validates credentials and signs a JWT containing ``{ id, role }`` using ``JWT_SECRET``; (3) Client stores JWT and sends it in ``Bearer`` header; (4) Server's ``protect`` middleware decodes and verifies the token on private requests.

20. How is email verification implemented?

Upon registration, a cryptographic token is generated, stored in ``user.verificationToken``, and emailed as an activation link via Nodemailer. Clicking the link calls ``/api/auth/verify/:token``, flipping ``isVerified = true``.

21. How do you prevent users from submitting claims on their own items?

In ``submitClaim``, the controller executes ``if (item.poster.toString() === req.user.id) return res.status(400).json({ msg: 'You cannot claim your own item.' })``.

22. How are concurrent or fake claims rejected?

When a finder approves a valid claim in ``verifyClaim``, all other pending claims on that item are atomically updated to ``status: 'rejected'``, and the item status flips to ``resolved``.

23. What is the difference between Free and Premium subscription plans?

Free plan users can post up to 12 items daily. Premium users have unlimited postings. Users apply for an upgrade via ``/plan``, and administrators review and approve requests in the Admin Dashboard.

24. How does the system handle Rate Limiting?

Using ``express-rate-limit``, we throttle API endpoints (e.g. max 5 login attempts per 15 minutes, daily item creation caps) to prevent brute-force login attacks and spamming.

25. If the platform scales to 1,000,000 items, what optimizations would you introduce?

1) Database compound indexes on ``{ category: 1, location: 1, status: 1 }``; 2) Redis caching for frequent feed queries; 3) CDN edge caching for Cloudinary media; 4) Elasticsearch for fuzzy text matching; 5) Kafka for async notification processing.

8. Examiner 'Trap Questions' & Smart Responses

Trap Question	Why Examiner Asks It	The Winning Answer Strategy
'Is JWT stored in localStorage safe from XSS?'	Tests knowledge of web security vulnerabilities.	Acknowledge: 'While localStorage is accessible via JavaScript (XSS risk), we sanitize inputs with express-validator and set CSP headers with Helmet. For banking-grade security, httpOnly SameSite cookies can be used as a next iteration.'
'Why not use WebSockets for notifications?'	Tests architectural trade-off reasoning.	Answer: 'For our current user load, REST-based polling with DB notifications keeps the server lightweight and stateless. Socket.io is earmarked for our next phase alongside real-time chat.'
'What happens if two people claim the same lost item?'	Tests your Double-Handshake understanding.	Answer: 'Both claims sit in 'pending' status with their private proof messages. The finder inspects both proofs, selects the genuine owner, and clicking Approve automatically rejects the rival claim.'

9. Live Project Demo Walkthrough Checklist

- **1. Registration & Verification:** Register a new user (``finder@test.com``) and show the verification email / direct activation.
- **2. Post Found Item:** Post a found item (e.g. 'Blue Leather Wallet at Kathmandu Mall') with image and a 500 NPR reward.
- **3. Post Lost Item:** Switch accounts or show explore search and filter by category ('Electronics') and location.

- **4. Submit Claim:** From a second account (`owner@test.com`), submit a claim with specific proof ('Has student ID ending in 402').
- **5. Finder Approval:** Log back into the finder account, show the private claim in dashboard, and click 'Approve Claim'.
- **6. Confirm Recovery:** Log into claimant account, click 'Confirm Recovery'. Show the item transitioning to 'Resolved'.
- **7. Badges & Hero Points:** Open the finder's profile and show +500 Hero Points and newly unlocked badges ('First Find' / 'Rising Hero').
- **8. Admin Dashboard:** Log in with admin credentials, show platform Recharts analytics, user approval table, and plan upgrade requests.

10. Golden Rules for Viva Day (Do's & Don'ts)

- **DO:** Speak clearly and confidently. Focus on **why** you built features, not just **what** they do.
- **DO:** Highlight the 'Double-Handshake Verification' — it's your primary unique selling point (USP).
- **DO:** Keep local servers running in the background (`npm run dev` on both frontend and backend) with MongoDB Atlas connected.
- **DON'T:** Say 'I don't know' abruptly. Instead, say: *'In our current implementation, we handle this via X; however, a great approach to optimize this would be Y.'*
- **DON'T:** Blame team members or tools if an error occurs. Stay calm, check terminal logs or browser DevTools console, and explain the troubleshooting step.